A SIKHAY × VALIGER COLLABORATION

in collaboration with

RADIAN Software

Project Description & GitHub Collaboration Package

Rotary · Angular · Display · Intuitive · Angle · Notation

July 13, 2026

Prepared by Sikhay Research, Development and Prototyping, in collaboration with Valiger
Author: Henry Gabriel Buban

Document Information

Document Title	RADIAN Software — Project Description & GitHub Collaboration Package
Version	1.0 (Draft)
Date	July 13, 2026
Author	Henry Gabriel Buban — Sikhay
Collaborator	Valiger
Scope	Companion application, firmware interface, and repository governance
Classification	Internal — Sikhay × Valiger Only

Project Description

Overview

The RADIAN Software Suite is the digital backbone of the Rotary Angular Display with Intuitive Angle Notation (RADIAN) — a co-developed educational hardware device produced by Sikhay Research, Development and Prototyping in collaboration with Valiger. The suite comprises two tightly coupled components: (1) firmware running on the ESP32 microcontroller embedded in the RADIAN device, and (2) a cross-platform mobile companion application that communicates with the device over Bluetooth Low Energy (BLE). Together, they transform the physical hardware into a fully interactive, lesson-aware teaching instrument.

The software is designed around a single core principle: the physical device and the digital application must feel like one object, not two products. When a student rotates the arm, the app responds in real time. When a teacher selects a lesson mode on the device, the app's visualization shifts without any additional interaction. The hardware instructs; the software amplifies.

Problem Statement

Secondary and early tertiary mathematics instruction in the Philippines relies predominantly on two tools that sit at opposite ends of the engagement spectrum: static physical manipulatives (protractors, printed charts, physical models) and free on-screen software (GeoGebra, Desmos, PhET). Physical tools have no feedback loop — a student can hold a protractor to 45° and never know if they are correct. Software tools have no physical grounding — students engage with pixels, not with real rotation.

Neither pole provides what the other lacks: physical grounding with real-time corrective feedback, connected to a live data source the teacher can monitor. RADIAN closes this gap. The software layer is what makes that closure possible — it is the feedback, the visualization, the quiz engine, and the teacher dashboard that the hardware alone cannot carry.

Target Users

- Secondary students (Grades 7–12) learning trigonometry, linear algebra, geometry, and introductory calculus
- Early tertiary students in STEM foundation courses encountering radians, vectors, and rotation matrices for the first time
- Mathematics and science teachers conducting hands-on demonstrations or structured lab activities
- School administrators evaluating and procuring educational technology for DepEd K-12 alignment

Scope of Software Deliverables

The RADIAN Software Suite covers the following deliverables within this collaboration:

- ESP32 firmware — angle sensing, mode logic, BLE data broadcasting, on-device OLED rendering
- Mobile companion application (Android primary, iOS secondary) — BLE connection management, live visualization, lesson modes, quiz engine, teacher dashboard
- BLE data contract specification — the shared message protocol that joins firmware and app
- GitHub repository structure, branching strategy, and collaboration governance documentation

The following are explicitly out of scope for this collaboration phase: web dashboard, school management system integration, cloud data storage, and backend API infrastructure.

Teaching Modes

Four teaching modes are committed for the initial release. Each maps to a distinct curriculum strand and drives a distinct visualization on the companion app.

Mode 1 — Degree / Radian Conversion

The device reads the primary arm's angle and displays it simultaneously in degrees and its radian equivalent. The companion app renders a full animated unit circle with the arm position, the arc length swept, and the (cos θ, sin θ) coordinate highlighted at the tip. This is the foundational mode and the original proof-of-concept from the Regional Mathematics Fair 2025.

Mode 2 — Vector Addition

Both arms are read independently, each treated as a vector with unit magnitude and direction equal to its angle. The firmware computes the resultant vector (sum of both) and broadcasts the component breakdown. The companion app displays an animated vector diagram showing both arms, their components, and the resultant tail-to-tip — the first mode to require the dual-arm mechanism.

Mode 3 — Rotation Matrix

The primary arm's angle θ defines a 2×2 rotation matrix R(θ). The firmware applies R to the secondary arm's position vector and broadcasts the transformed coordinates. The companion app renders the before/after transformation live, making the geometric effect of matrix multiplication physically observable — a concept that is notoriously abstract when taught purely symbolically.

Mode 4 — Polygon / Central Angle Snap

The user selects a polygon order N (3 through 12) via the device buttons. The arm snaps to the nearest multiple of 360°/N and displays interior, central, and exterior angle values for that N-gon. The companion

app renders the full regular polygon inscribed in a circle, with the current vertex highlighted. This mode teaches the angle geometry of regular polygons and the relationship between central and interior angles.

Expansion Backlog (Post-Initial Release)

- Quiz mode — teacher-issued angle targets, student arm-matching, scored response logging
- Teacher dashboard — BLE-aggregated per-student accuracy and response-time data during class
- Wave-trace mode — real-time sin/cos curve traced as the student sweeps the arm
- Cartesian grid plate support — slope, y-intercept, and linear function mode
- Edge slider integration — live parameter adjustment for function graphing
- Multi-device BLE management — teacher app managing multiple student RADIAN units

Technical Requirements & Stack

Firmware — ESP32

Platform: ESP32-WROOM-32 DevKit V1

Framework: Arduino framework via PlatformIO (VSCode extension)

Language: C++ (Arduino dialect)

Sensors: AS5600 magnetic rotary encoder × 2 (I²C, 0x36 / 0x37 via TCA9548A I²C mux for dual-arm)

Display: SSD1306/SH1106 OLED, 0.96" — I²C

BLE Library: ESP32 BLE Arduino (NimBLE preferred for lower memory footprint)

Key Libraries: Wire.h (I²C), AS5600 library, Adafruit_SSD1306, ArduinoJson (BLE payload serialization)

BLE Data Contract

The BLE contract is the single most critical shared deliverable — firmware and app must agree on it before parallel development can begin. It is defined here and must not be changed unilaterally by either track.

```
Service UUID:       4A2B-RADIAN-0001 (custom, 128-bit)
Characteristic:     ANGLE_DATA  (notify, read)

Payload (JSON over BLE notify, ≤ 100 bytes):
{
  "mode": 1,          // active teaching mode (1–4)
  "a1": 47.3,         // arm 1 angle in degrees
  "a2": 112.8,        // arm 2 angle in degrees (0.0 if single-arm mode)
  "val": {            // computed mode-specific values
    "rad": 0.825,     // mode 1: radian equivalent
    "rx": 0.682,      // modes 1/3: cos θ
    "ry": 0.731,      // modes 1/3: sin θ
    "rmag": 1.41,     // mode 2: resultant magnitude
```

```
    "rang": 80.05,    // mode 2: resultant angle
    "snap": 6,        // mode 4: polygon N
    "int": 120.0,     // mode 4: interior angle
    "ext": 60.0       // mode 4: exterior angle
  },
  "ts": 1720863600    // unix timestamp from ESP32 millis()
}
```

Mobile Companion App

Framework: Flutter 3.x (Dart)

Target Platforms: Android 8.0+ (primary), iOS 13+ (secondary)

BLE Library: flutter_blue_plus

State Management: Riverpod (Provider as fallback if Riverpod is unfamiliar to contributors)

Visualization: fl_chart (charts), custom Canvas/CustomPainter (unit circle, vector diagram, polygon)

Navigation: go_router

Local Storage: Hive (lightweight, no server needed for pilot)

Min SDK: Android API 26 / iOS 13

App Screen Architecture

- Scan & Connect Screen — discover nearby RADIAN devices by BLE name/UUID, display signal strength, connect with single tap
- Home / Mode Screen — mirrors active mode from device; allows mode switch from app; shows live angle readout badge
- Mode 1 Visualizer — animated unit circle, sweeping arc, cos/sin coordinate panel, degree↔radian dual display
- Mode 2 Visualizer — two-vector diagram with component breakdown, resultant vector, magnitude/angle readout
- Mode 3 Visualizer — 2×2 rotation matrix display, before/after transformed vector, animated rotation arc
- Mode 4 Visualizer — inscribed regular polygon, highlighted current vertex, interior/exterior/central angle panel
- Settings Screen — device nickname, unit preference (degrees/radians), theme, connection management

Repository Structure

The project lives in a single GitHub organization repository under Sikhay's account, with two top-level packages — firmware and app — that are developed by separate tracks but share the BLE contract as a single source of truth.

```
radian/
├── firmware/                  # ESP32 PlatformIO project
│   ├── src/
│   │   ├── main.cpp
│   │   ├── ble/
│   │   │   ├── BLEService.cpp
│   │   │   └── BLEService.h
│   │   ├── modes/
│   │   │   ├── ModeManager.cpp
│   │   │   ├── Mode1_DegRad.cpp
│   │   │   ├── Mode2_VectorAdd.cpp
│   │   │   ├── Mode3_RotMatrix.cpp
│   │   │   └── Mode4_PolygonSnap.cpp
│   │   └── sensors/
│   │       ├── EncoderA.cpp
│   │       └── EncoderB.cpp
│   ├── include/
│   ├── platformio.ini
│   └── test/
├── app/                       # Flutter project
│   ├── lib/
│   │   ├── main.dart
│   │   ├── ble/
│   │   │   ├── ble_manager.dart
│   │   │   └── radian_packet.dart   # BLE payload model
│   │   ├── screens/
│   │   ├── widgets/
│   │   │   └── painters/            # CustomPainter visualizations
│   │   ├── models/
│   │   └── providers/               # Riverpod
│   ├── android/
│   ├── ios/
│   └── pubspec.yaml
├── docs/
│   ├── ble_contract.md
│   ├── hardware_pinout.md
│   └── architecture.md
├── .github/
│   ├── ISSUE_TEMPLATE/
```

```
│   │   ├── bug_report.md
│   │   └── feature_request.md
│   └── workflows/
│       └── flutter_ci.yml
├── README.md
├── CONTRIBUTING.md
└── LICENSE
```

Branching Strategy

main: stable, production-ready code only — direct push prohibited, PR + review required

dev: integration branch — all feature branches merge here first; weekly sync with main

feature/firmware-*: firmware feature branches (e.g., feature/firmware-mode2-vector)

feature/app-*: app feature branches (e.g., feature/app-mode1-unitcircle-painter)

fix/*: hotfix branches off main

docs/*: documentation-only changes

Pull requests require at least one reviewer approval. The BLE contract (ble_contract.md and radian_packet.dart) requires sign-off from both the firmware lead and the app lead before merge.

Development Environment Requirements

Firmware Track

- Visual Studio Code + PlatformIO IDE extension
- PlatformIO Core (installed via extension)
- USB-to-serial driver (CP210x or CH340 depending on ESP32 board variant)
- Python 3.x (PlatformIO dependency)
- Git 2.x

App Track

- Flutter SDK 3.x (stable channel)
- Android Studio or VSCode with Flutter/Dart extensions
- Android SDK API 26+
- Physical Android device for BLE testing (emulators cannot simulate BLE)
- Git 2.x

GitHub Collaboration Documents

The following section contains the ready-to-paste text for each required GitHub repository file. Copy each block into the corresponding file in the repository.

README.md

````
# RADIAN — Rotary Angular Display with Intuitive Angle Notation

A co-development by **Sikhay Research, Development and Prototyping** × **Valiger**.

RADIAN is an ESP32-powered educational device that teaches abstract mathematics
through physical rotation. This repository contains the companion mobile
application
(Flutter) and the ESP32 firmware (PlatformIO / Arduino).

## What it teaches

| Mode | Concept |
|------|---------|
| 1 — Degree / Radian | Unit circle, sin/cos coordinates, arc length |
| 2 — Vector Addition | Component breakdown, resultant, magnitude |
| 3 — Rotation Matrix | 2×2 rotation, geometric transformation |
| 4 — Polygon Snap    | Interior, exterior, and central angles |

## Repository layout

```
radian/
├── firmware/   ESP32 PlatformIO project
├── app/        Flutter companion application
└── docs/       BLE contract, pinout, architecture
```

## Quick start

### Firmware
```bash
cd firmware
pio run --target upload
```

### App
```bash
````
````

````
cd app
flutter pub get
flutter run
```

## BLE contract

The BLE message format is the single source of truth shared by firmware and app.
Any change requires approval from both tracks. See `docs/ble_contract.md`.

## Contributing

See [CONTRIBUTING.md](./CONTRIBUTING.md) for branch rules, commit format,
and pull request requirements.

## License

Proprietary — © 2026 Sikhay Research, Development and Prototyping / Valiger.
All rights reserved. See [LICENSE](./LICENSE).
````

CONTRIBUTING.md

````
# Contributing to RADIAN

This repository is a closed collaboration between Sikhay and Valiger.
External contributions are not accepted at this stage.

## Tracks and ownership

| Track    | Owner   | Scope |
|----------|---------|-------|
| Firmware | Sikhay  | ESP32, PlatformIO, BLE broadcast |
| App      | Valiger | Flutter, BLE receive, visualizations |
| BLE contract | Both | Shared sign-off required |

## Branch naming

```
feature/firmware-<short-description>
feature/app-<short-description>
````

````
fix/<short-description>
docs/<short-description>
```

## Commit message format

```
<type>(<scope>): <short summary>

Types: feat | fix | docs | refactor | test | chore
Scope: firmware | app | ble | docs

Examples:
  feat(firmware): add Mode 2 vector resultant computation
  fix(app): resolve BLE disconnect crash on Android 12
  docs(ble): clarify ts field as millis() not epoch
```

## Pull request rules

- All PRs target `dev` (never `main` directly).
- Minimum 1 reviewer approval required.
- BLE contract changes require both firmware lead
  and app lead approval.
- Include a short test note (what you tested, on
  what device/board).

## Weekly sync

`dev` is merged into `main` every week after a
joint smoke-test. The firmware lead and app lead
must both confirm the build passes before merge.
````

docs/ble_contract.md

```
# BLE Contract — RADIAN

**Version:** 1.0   |   **Status:** Locked for v1 — changes require dual approval

## Service
```
```

````
| Property | Value |
|----------|-------|
| Service UUID | `4A2B-RADIAN-0001` (custom 128-bit) |
| Characteristic | `ANGLE_DATA` |
| Properties | Notify, Read |
| Notify interval | ~50 ms (20 Hz) |

## Payload

JSON string, UTF-8 encoded, ≤ 100 bytes.

```json
{
  "mode": 1,
  "a1": 47.3,
  "a2": 112.8,
  "val": {
    "rad": 0.825,
    "rx": 0.682,
    "ry": 0.731,
    "rmag": 1.41,
    "rang": 80.05,
    "snap": 6,
    "int": 120.0,
    "ext": 60.0
  },
  "ts": 1720863600
}
```

## Field definitions

| Field | Type | Description |
|-------|------|-------------|
| mode  | int  | Active mode (1–4) |
| a1    | float | Arm 1 angle, degrees, 0–359.9 |
| a2    | float | Arm 2 angle, degrees; 0.0 if single-arm mode |
| val.rad  | float | Mode 1: radian equivalent of a1 |
| val.rx   | float | Mode 1/3: cos(a1) |
| val.ry   | float | Mode 1/3: sin(a1) |
````

```
| val.rmag | float | Mode 2: resultant vector magnitude |
| val.rang | float | Mode 2: resultant angle, degrees |
| val.snap | int   | Mode 4: selected polygon N |
| val.int  | float | Mode 4: interior angle of N-gon |
| val.ext  | float | Mode 4: exterior angle of N-gon |
| ts    | long  | ESP32 millis() at time of reading |

## Change process

1. Open a GitHub issue tagged `ble-contract`.
2. Both firmware lead and app lead comment approval.
3. PR updates this file AND `app/lib/ble/radian_packet.dart`
   in the same commit.
```

Bug Report Issue Template (.github/ISSUE_TEMPLATE/bug_report.md)

```
---
name: Bug Report
about: Something is broken in firmware or app
labels: bug
---

## Track
- [ ] Firmware (ESP32)
- [ ] App (Flutter)
- [ ] BLE connection

## Description
<!-- Clear description of what is wrong -->

## Steps to reproduce
1.
2.
3.

## Expected behaviour

## Actual behaviour

## Environment
- Board / device: (e.g., ESP32-WROOM-32 / Samsung A32 Android 12)
```

```
- Firmware version / App version:
- Mode when bug occurred:

## Screenshots / serial logs
<!-- Attach if available -->
```

Feature Request Issue Template (.github/ISSUE_TEMPLATE/feature_request.md)

```
---
name: Feature Request
about: Propose a new feature or enhancement
labels: enhancement
---

## Track
- [ ] Firmware
- [ ] App
- [ ] BLE contract change (requires dual approval)

## Problem this solves
<!-- What teaching need or workflow issue does this address? -->

## Proposed solution

## Alternatives considered

## Teaching mode affected
- [ ] Mode 1 — Degree/Radian
- [ ] Mode 2 — Vector Addition
- [ ] Mode 3 — Rotation Matrix
- [ ] Mode 4 — Polygon Snap
- [ ] New mode
- [ ] General / cross-cutting

## Additional context
```

CI Workflow (.github/workflows/flutter_ci.yml)

```
name: Flutter CI


on:
```

```
  push:
    branches: [ dev, main ]
    paths: [ 'app/**' ]
  pull_request:
    branches: [ dev, main ]
    paths: [ 'app/**' ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.x'
          channel: stable

      - name: Install dependencies
        working-directory: app
        run: flutter pub get

      - name: Analyze
        working-directory: app
        run: flutter analyze

      - name: Run tests
        working-directory: app
        run: flutter test

      - name: Build APK (debug)
        working-directory: app
        run: flutter build apk --debug
```

LICENSE

```
This software and associated documentation files (the 'Software') are
the proprietary and confidential property of Sikhay Research,
Development and Prototyping and Valiger. No part of the Software may
be reproduced, distributed, modified, or used in any form or by any
means without the prior written permission of both parties.

THE SOFTWARE IS PROVIDED 'AS IS', WITHOUT WARRANTY OF ANY KIND.
IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR
OTHER LIABILITY ARISING FROM THE USE OF THE SOFTWARE.
```

.gitignore (recommended entries)

```
# PlatformIO
.pio/
.pioenvs/
firmware/.pioenvs/

# Flutter
app/.dart_tool/
app/build/
app/.flutter-plugins
app/.flutter-plugins-dependencies

# IDE
.vscode/settings.json
.idea/
*.swp

# OS
.DS_Store
Thumbs.db
```

Milestone Plan

This is a working timeline organized around the BLE contract as the hard synchronization gate between the two tracks. Dates are relative to project start.

Gate	Timeline	Deliverable
M0	Week 1	Repository setup, branch structure, agreed development environments — both tracks
M1	Week 2	BLE contract v1.0 locked (ble_contract.md signed off by both leads) — the gate that unlocks parallel dev

M2	Week 3–4	Firmware: Modes 1 & 2 broadcasting over BLE | App: BLE scanner + mock-data Mode 1 visualizer running
M3	Week 5–6	Firmware: Modes 3 & 4 complete | App: all four mode visualizers running on mock data
M4	Week 7	Full integration: app connects to real hardware, all four modes validated end-to-end
M5	Week 8	Pilot build: APK distributed to pilot teacher, firmware flashed on prototype unit, feedback collection begins